Java Programming Guide

From Basic Syntax to Object-Oriented Concepts

1. The Anatomy of a Java Program & Printing Output

Every Java program must have at least one **Class** and a **Main Method**. The `main` method is the critical entry point where the Java Virtual Machine (JVM) starts executing your code.

To print output to the console, Java provides the built-in `System.out.println()` statement.

```
// Every Java file must have a public class matching the filename (e.g., Main.j
public class Main {

    // The main method: execution starts here
    public static void main(String[] args) {

        // println prints text and automatically moves to a new line
        System.out.println("Hello, World!");

        // print outputs text without moving to a new line
        System.out.print("Learning Java ");
        System.out.print("is fun!");
    }
}
```

2. Variables and Data Types

A variable acts as a container holding a specific type of data. Java is a "strongly-typed" language, which means you must explicitly declare the data type a variable will store before using it.

Declaration and Assignment

Variables can be declared and assigned values in separate steps or combined into a single initialization statement.

```
// 1. Declare first, assign later
int age;
age = 25;

// 2. Declare and assign in one line (Initialization)
String name = "Alex";
```

Common Data Types

Type	Description	Common Usage	Example
`int`	Whole numbers	Counting, IDs, general numbers	`int score = 100;`
`double`	Decimal numbers	Prices, precise measurements	`double price = 19.99;`
`char`	Single character	Grades, specific symbols	`char grade = 'A';`
`boolean`	True or False	Flags, states, conditions	`boolean isOnline = true;`
`String`	Text	Names, messages, sentences	`String city = "Lahore";`

3. Getting User Input

To interactively get input from a user via the console, Java provides the `Scanner` class within the `java.util` package. You must import this class before utilizing it.

```
import java.util.Scanner; // 1. Import the Scanner class

public class InputExample {
    public static void main(String[] args) {

        // 2. Create a Scanner object connected to System.in (the keyboard)
        Scanner scanner = new Scanner(System.in);

        System.out.println("What is your name?");
        // 3. Read the user's input as a String
        String userName = scanner.nextLine();

        System.out.println("How old are you?");
        // 4. Read the user's input as an integer
        int userAge = scanner.nextInt();

        System.out.println("Hello " + userName + ", you are " + userAge + " yea

        // 5. Always close the scanner when done to free up resources
        scanner.close();
    }
}
```

4. Control Flow (Conditionals)

Conditionals are essential for programming logic, allowing your application to make decisions and execute different code blocks based on varying conditions using `if`, `else if`, and `else`.

```
int temperature = 30;

if (temperature > 35) {
    System.out.println("It's extremely hot!");
} else if (temperature >= 20) {
    // This block will execute because 30 is >= 20
    System.out.println("The weather is nice.");
} else {
    System.out.println("It's cold.");
}
```

5. Loops

Loops provide a mechanism to execute a block of code repeatedly, saving you from writing repetitive statements.

The `for` Loop

The `for` loop is ideal when you know **exactly how many times** you want the loop to iterate.

```
// Structure: for (initialization; condition; increment)
for (int i = 1; i <= 5; i++) {
    System.out.println("Count is: " + i);
}
```

The `while` Loop

The `while` loop is best used when you want a block of code to run continuously **until a specific condition becomes false**.

```
int count = 1;
while (count <= 3) {
    System.out.println("While loop count: " + count);
    count++; // Critical: Must increment to avoid an infinite loop
}
```

6. Object-Oriented Programming (OOP) Concepts

Java is fundamentally an Object-Oriented language. The core philosophy is to model real-world entities and concepts as structural components in code using **Classes** and **Objects**.

Classes and Objects Explained

- **Class:** The blueprint, template, or design. It defines the state (attributes/variables) and behavior (methods/functions) that an object of this type will possess.
- **Object:** A specific, tangible instance instantiated (created) from that class blueprint.

> 💡 *Think of a Class as the architectural blueprints for a house, and an Object as the actual physical house built from those blueprints.*

Understanding Constructors

A constructor is a special type of method invoked specifically to initialize newly created objects.

- It executes automatically when an object is created using the `new` keyword.
- It **must** have the exact same name as the Class it resides in.
- It does **not** have a return type, not even `void`.

Putting It All Together: Classes, Constructors, and Objects

Below is a comprehensive example demonstrating how to define a `Car` class, establish a constructor, and instantiate objects in a main program.

```
// 1. Define the Class (The Blueprint)
class Car {

    // Attributes (Instance Variables defining state)
    String brand;
    String color;
    int year;

    // 2. Define the Constructor
    // Executes automatically upon calling 'new Car(...)'
    public Car(String brandName, String carColor, int carYear) {
        brand = brandName;
        color = carColor;
        year = carYear;
    }

    // 3. Define a Method (Behavior)
    public void startEngine() {
        System.out.println("The " + color + " " + brand + " starts with a roar!");
    }
}

// 4. The Main Class to execute the application
public class Dealership {
    public static void main(String[] args) {

        // 5. Creating Objects (Instantiating the Class)
        // Using the 'new' keyword invokes the constructor
        Car myFirstCar = new Car("Toyota", "Red", 2021);
        Car mySecondCar = new Car("Honda", "Black", 2023);

        // 6. Interacting with the Objects
        System.out.println("My first car is from " + myFirstCar.year);

        // Calling methods on specific object instances
        myFirstCar.startEngine();
        mySecondCar.startEngine();
    }
}
```